EXPERIMENT 6

End-to-End Data Pipeline Orchestration using Azure Data Factory and Azure Databricks

Aim

To integrate Azure Data Factory with Azure Databricks for end-to-end data pipeline orchestration, where ADF triggers a Databricks notebook to preprocess and transform data ingested from Azure Data Lake Storage Gen2.

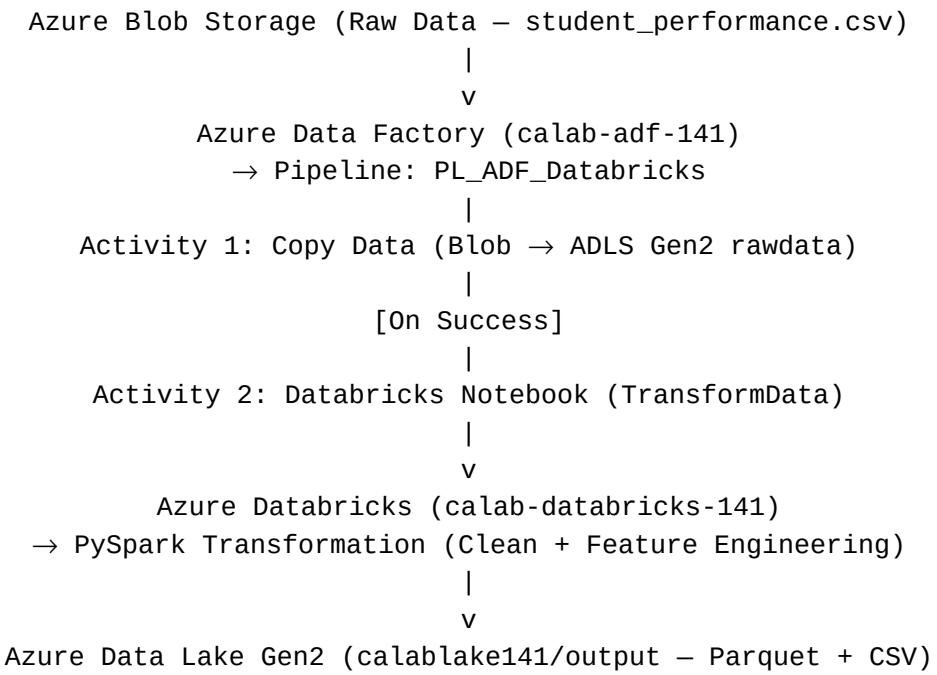
Theory

Azure Data Factory (ADF) is a cloud-based data orchestration service that enables the creation of automated data pipelines. While ADF handles data movement and scheduling, it can also trigger compute workloads in Azure Databricks using the Databricks Notebook activity. This integration enables a fully automated end-to-end pipeline covering ingestion, transformation, and storage.

Azure Databricks provides a collaborative Apache Spark environment for large-scale data processing and machine learning. When integrated with ADF, Databricks notebooks can be executed as part of an ADF pipeline, enabling complex transformations such as data cleaning, feature engineering, and aggregation to be performed on raw data fetched from Data Lake Storage.

In this experiment, ADF orchestrates a pipeline that first ingests raw student data (Abhi, Kshitiz, Aakriti, Rohit, Sandhya, Riya) from Azure Blob Storage into Data Lake Gen2, and then triggers a Databricks notebook to preprocess and transform the data. The transformed output is written back to the Data Lake for downstream consumption.

Flow Diagram



Procedure

1. Open Azure Data Factory Studio for calab-adf-141.

2. Navigate to Manage → Linked Services → New → Azure Databricks.

3. Configure the Databricks linked service using the workspace URL (adb-7405609713574305.5.azuredatabricks.net) and MSI authentication.

4. Select New Job Cluster configuration: Runtime 14.3, Standard_DS3_v2, 1 worker.

5. In Azure Databricks, import the TransformData.py notebook with PySpark logic for data cleaning and feature engineering.

6. In ADF Studio → Author → Pipelines → New Pipeline → name it PL_ADF_Databricks.

7. Drag Copy Data activity onto canvas → set source as Blob Storage (DS_BlobSource_CSV), sink as Data Lake rawdata container (DS_ADLS_Sink_CSV).

8. Drag Databricks Notebook activity → connect it after Copy Data activity (success dependency).

9. Configure Databricks Notebook activity → select LS_Databricks linked service → browse and select TransformData notebook.

10. Click Validate → verify no errors → Publish All.

11. Click Add Trigger → Trigger Now to run the pipeline.

12. Monitor → Pipeline Runs → verify both Copy Data and Databricks Notebook activities show Succeeded status.

13. Verify transformed Parquet output in calablake141/output/student_performance_cleaned and summary CSV in output/summary_stats.

Expected Output

<GitHub Link Here>

Result

Thus, an end-to-end data pipeline was successfully orchestrated using Azure Data Factory and Azure Databricks. ADF automated the ingestion of raw data from Blob Storage to Data Lake Gen2 and triggered a Databricks notebook to perform PySpark-based data transformation. The cleaned and processed output was written back to Data Lake in Parquet format with summary statistics in CSV. This experiment demonstrates a modern cloud-native ELT architecture used in enterprise data engineering for scalable and automated data workflows.